

Universidad Nacional de Río Cuarto

Licenciatura en Ciencias de la Computación

Taller de Diseño de Software

Preproyecto

Implementación de un intérprete

Alumnos: Francisco Natale, Agustin Nahuel Alieni, Julian Varea

1. Flujo general del procesamiento

La implementación comienza con el análisis léxico del código fuente. Para esta etapa se utilizó un *lexer*, cuyo objetivo es reconocer los **tokens** a partir de los lexemas que aparecen en el código.

Por ejemplo, a partir de una sentencia como:

```
int var;
```

el lexer permite reconocer elementos como la palabra reservada **int**, el identificador **var** y el símbolo **;**.

Una vez obtenidos estos tokens, se utiliza Bison para reconocer las **estructuras sintácticas** definidas por la gramática. De esta forma, los tokens reconocidos por el lexer comienzan a organizarse en estructuras que representan declaraciones, asignaciones, expresiones y demás construcciones del lenguaje.

A medida que Bison reconoce estas estructuras, se construyen los nodos que formarán el árbol sintáctico abstracto (AST). Por ejemplo, una declaración como:

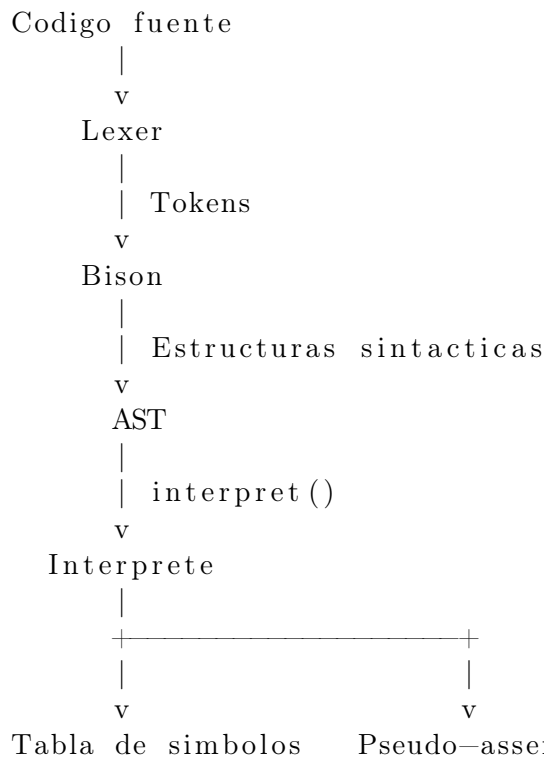
```
int var;
```

se transforma en un nodo de declaración, mientras que:

```
var = 10;
```

se transforma en un nodo de asignación que contiene una expresión representada por un nodo constante.

De esta manera, el proceso puede resumirse como:



A partir de este punto, el árbol deja de ser solamente una representación de la estructura del programa y pasa a ser utilizado para su ejecución. El intérprete recorre los nodos del AST, evalúa las expresiones, actualiza la tabla de símbolos y genera el pseudo-assembly correspondiente.

Por lo tanto, de forma conceptual, las etapas pueden verse como:

Lexer → Bison → AST → Intérprete

El lexer permite identificar los elementos del código, Bison permite reconocer cómo se relacionan sintácticamente y, finalmente, el intérprete “le da vida” a la estructura obtenida al ejecutar las sentencias y evaluar sus expresiones.

2. Construcción del AST

A medida que Bison reconoce las diferentes reglas de la gramática, se construyen los nodos correspondientes del árbol.

Por ejemplo, para la declaración:

```
int var;
```

se construye un nodo de tipo **VariableDeclaration**, que contiene el tipo de la variable y su identificador:

```
VariableDeclaration
    type = INT
    id = "var"
```

Por otro lado, para la asignación:

```
var = 10;
```

se construye un nodo de tipo **VariableAssignment**:

```
VariableAssignment
    id = "var"
    expression = Node_Number(10)
```

La expresión correspondiente al número 10 se representa mediante un nodo propio. De esta manera, la asignación no almacena solamente el texto “10”, sino una referencia a un nodo del árbol que representa dicha expresión.

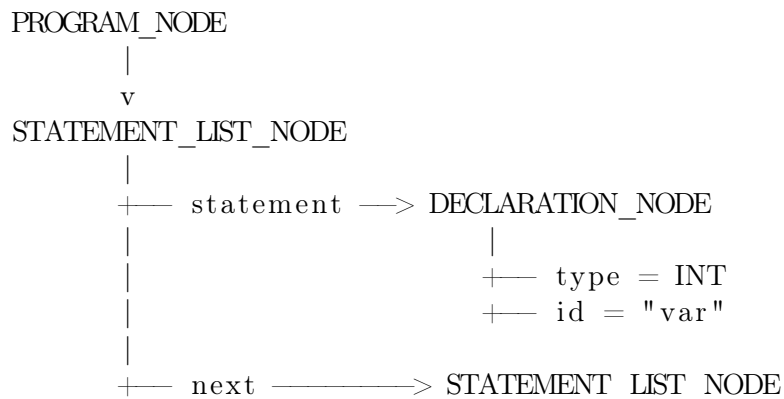
2.1. Lista de sentencias

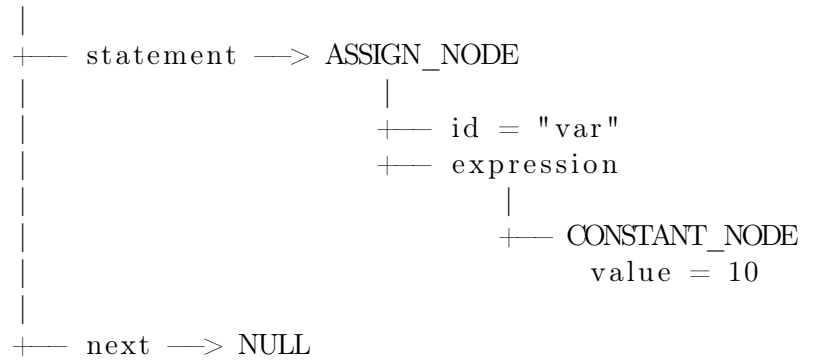
En la implementación, las sentencias se organizan mediante una lista enlazada representada por nodos **STATEMENT_LIST_NODE**. Cada nodo contiene una sentencia y una referencia a la siguiente sentencia.

Por ejemplo, el programa:

```
void main() {
    int var;
    var = 10;
}
```

puede representarse conceptualmente como:





3. Implementación del intérprete

Para la implementación del intérprete se recorrió la lista de nodos generadas al procesar el código fuente, se utilizó una tabla de símbolos que almacena las variables que aparecen durante la ejecución del programa, junto con su tipo y su valor actual.

Las principales situaciones consideradas son:

1. Asignaciones

Cuando se encuentra una asignación, primero se evalúa recursivamente la expresión ubicada a la derecha del operador “=” y, una vez obtenido su valor, se actualiza la entrada correspondiente en la tabla de símbolos.

Por ejemplo:

$$x = 2 + 3$$

Primero se evalúa la expresión $2 + 3$, obteniendo el valor 5. Luego se actualiza la variable x en la tabla de símbolos:

$$x \rightarrow 5$$

2. Operaciones binarias

Cuando se encuentra una operación binaria, como una suma, resta o multiplicación, sus operandos se evalúan recursivamente.

Por ejemplo, para la expresión:

$$2 + 3$$

se reduce primero el hijo izquierdo y luego el hijo derecho. Una vez obtenidos ambos valores, se realiza la operación:

$$2 + 3 = 5$$

Este mecanismo se aplica recursivamente para expresiones más complejas. Por ejemplo:

$$(2 + 3) * 4$$

primero se reduce la expresión interna:

$$2 + 3 = 5$$

y posteriormente se utiliza ese resultado para resolver:

$$5 * 4 = 20$$

3. Sentencias `return`

En el caso de una sentencia `return`, se evalúa la expresión asociada.

Si se retorna una variable, se consulta su valor en la tabla de símbolos. Si se retorna una constante, se utiliza directamente su valor. También se contempla el caso de un `return` sin expresión, que representa un retorno `void`.

Por ejemplo:

```
return x
```

implica buscar el valor actual de x en la tabla de símbolos.

4. Generación del pseudo-assembly

Además de evaluar las expresiones, el intérprete genera un *pseudo-assembly* de tres direcciones para representar las operaciones realizadas.

En el caso de las operaciones binarias, las expresiones se resuelven de forma recursiva. Cada vez que se obtiene el resultado de una operación se utiliza una variable temporal, representada mediante nombres como $T1$, $T2$, etc.

El mecanismo utilizado para comunicar la ubicación donde quedó el resultado de una expresión es la variable `last_expr_loc`.

La función `evaluate_expression()` tiene, conceptualmente, dos resultados:

1. El **valor numérico** de la expresión, retornado mediante `return`.
2. La **ubicación del resultado** dentro del pseudo-assembly, almacenada en `last_expr_loc`.

Por ejemplo, al evaluar:

$2 + 3$

se genera:

```
SUM 2 3 T1
```

En este punto se tienen dos informaciones diferentes:

```
valor = 5
```

```
last_expr_loc = "T1"
```

El valor 5 permite continuar con la interpretación del programa, mientras que $T1$ permite continuar generando el pseudo-assembly.

5. Resolución de una operación binaria

Consideremos una operación binaria cuyos dos operandos son constantes:

$2 + 3$

El procedimiento puede resumirse en los siguientes pasos:

1. Se evalúa recursivamente el operando izquierdo.

2

Se obtiene:

```
valor = 2
```

```
ubicacion = "2"
```

2. Se guarda la ubicación del operando izquierdo para evitar que sea sobrescrita al evaluar el siguiente hijo:

```
left_loc = "2"
```

3. Se evalúa recursivamente el operando derecho:

```
3
```

obteniendo:

```
valor = 3
ubicacion = "3"
```

4. Se genera una nueva variable temporal para almacenar el resultado:

```
T1
```

5. Se genera la instrucción de tres direcciones:

```
SUM 2 3 T1
```

6. Se actualiza `last_expr_loc` para indicar que el resultado de la expresión se encuentra ahora en T1:

```
last_expr_loc = "T1"
```

7. Finalmente, la función devuelve el resultado numérico:

```
return 5
```

6. Expresiones binarias anidadas

Cuando los operandos no son constantes simples, el mismo procedimiento se aplica recursivamente tantas veces como operaciones binarias existan en la expresión.

Por ejemplo:

```
x = (2 + 3) * 4
```

La evaluación comienza por la expresión más interna:

```
2 + 3
```

que genera:

```
SUM 2 3 T1
```

y deja:

```
last_expr_loc = "T1"
```

Este resultado pasa a ser el operando izquierdo de la siguiente operación. Luego se evalúa:

```
T1 * 4
```

generando:

```
MUL T1 4 T2
```

Finalmente:

```
last_expr_loc = "T2"
```

y el resultado numérico de toda la expresión es 20.

Al volver al nodo de asignación se tienen ambas informaciones:

```
valor = 20
ubicacion = "T2"
```

Por lo tanto, se actualiza la tabla de símbolos:

```
x -> 20
```

y se genera:

```
ASIG T2 x
```

El pseudo-assembly completo resulta:

```
SUM 2 3 T1
MUL T1 4 T2
ASIG T2 x
```

7. Función de `last_expr_loc`

La variable `last_expr_loc` permite transportar hacia el nodo padre la ubicación donde quedó almacenado el resultado de una expresión.

Esto es necesario debido a que `evaluate_expression()` devuelve el valor numérico de la expresión, pero este valor por sí solo no permite saber cómo referenciar el resultado al generar el pseudo-assembly.

Por ejemplo, para:

```
(2 + 3) * 4
```

al resolver:

```
2 + 3
```

se obtiene:

```
valor = 5
last_expr_loc = "T1"
```

El valor 5 es utilizado por el intérprete, mientras que T1 es utilizado para generar la siguiente operación:

```
MUL T1 4 T2
```

Por esta razón, durante la evaluación de una operación binaria se guardan también las ubicaciones de los operandos:

```
left_loc
right_loc
```

Esto evita perder la ubicación del primer operando cuando se evalúa recursivamente el segundo, ya que cada nueva evaluación actualiza `last_expr_loc`.

En consecuencia, el mecanismo puede resumirse como:

valor numérico → interpretación

`last_expr_loc` → generación de pseudo-assembly

De esta manera, la evaluación recursiva del AST permite realizar simultáneamente dos tareas: obtener el resultado de la expresión para la ejecución del programa y conservar la ubicación del resultado para generar el pseudo-assembly correspondiente.